

Informe Técnico - ComercioTech

Implementación de una Base de Datos NoSQL con MongoDB

NOMBRE: Fernanda De La Mora Díaz.

CARRERA: Analista Programador.

ASIGNATURA: Base de Datos No Estructuradas.

PROFESOR: Paul Beltrand.

Índice

Contenido

Informe Técnico - ComercioTech	1
Implementación de una Base de Datos NoSQL con MongoDB	1
Índice	2
Introducción	4
Descripción del caso de estudio.....	4
Objetivo General	5
Objetivos específicos.....	5
Diagnostico e identificación	5
<i>Diagnóstico de la situación actual</i>	5
<i>Requisitos de rendimiento</i>	5
<i>Requisitos de escalabilidad</i>	6
<i>Requisitos de disponibilidad</i>	6
<i>Requisitos de seguridad</i>	6
<i>Conclusión del diagnóstico</i>	6
Requisitos del sistema.....	6
<i>Requisitos funcionales</i>	6
<i>Requisitos no funcionales</i>	7
<i>Usuarios del sistema</i>	7
<i>Restricción del proyecto</i>	7
Selección y justificación del Sistema Operativo	8
<i>Sistema operativo seleccionado</i>	8
<i>Justificación de la elección</i>	8
Configuración del entorno virtualizado.....	9
<i>Entorno utilizado</i>	9
<i>Recursos asignados</i>	9



Configuración realizada 10

Instalación y configuración de MongoDB 10

Instalación de MongoDB..... 10

Diseño de la Base de Datos..... 11

Modelo documental..... 11

Colección Customers..... 11

Colección Products..... 11

Colección Orders..... 11

Evidencias 12

Introducción

El siguiente informe técnico tiene como propósito diseñar e implementar una solución de base de datos no estructurada para la empresa ficticia ComercioTech, organización que se encuentra en proceso de expansión y necesita modernizar su infraestructura tecnológica. La empresa utiliza un sistema heredado que presenta limitaciones de rendimiento, escalabilidad y soporte para nuevas funcionalidades.

Como solución, se propone la implementación de MongoDB como sistema gestor de base de datos, esto por su modelo documental, flexibilidad y capacidad de escalamiento. Ésta permitirá administrar información en grandes volúmenes de clientes, productos y pedidos, además de facilitar la integración con una aplicación desarrollada para poder realizar operaciones de creación, lectura, actualización y eliminación de datos.

Durante el desarrollo del proyecto se abordarán las etapas de análisis de requisitos, selección y configuración del sistema operativo, implementación de un entorno virtualizado, instalación y configuración segura del DBMS, diseño de las colecciones, desarrollo de componentes de software e información de la base de datos con registros de prueba.

Finalmente, se analizarán los aspectos de seguridad, normativa y ética relacionados con la gestión de información, considerando la protección de los datos personales y la aplicación de buenas practicas en el desarrollo de la solución.

Descripción del caso de estudio

ComercioTech es una empresa dedicada a la comercialización de productos y actualmente se encuentra en un proceso de crecimiento. Este aumento en sus operaciones ha significado también un incremento en sus clientes, productos registrados y transacciones comerciales.

La empresa utiliza una base de datos heredada que no responde adecuadamente a sus nuevas necesidades. Entre las principales dificultades está la baja capacidad de escalamiento, tiempos de respuesta poco eficientes, problemas para agregar nuevas funcionalidades y limitaciones en la administración de la información.

Gracias a los problemas que arrastra una actualización de su infraestructura de base de datos de manera urgente, la cual permita gestionar de forma centralizada y segura los siguientes elementos:

- Información de clientes.
- Catálogo de productos.
- Control de stock.
- Registro de pedidos.
- Detalle de productos incluidos en cada pedido.
- Estado de los pedidos.
- Historial de transacciones.
- La solución propuesta estará basada en MongoDB, un sistema de gestor de base de datos NoSQL, orientado a documentos. Se diseñarán colecciones para clientes, productos y pedidos, utilizando documentos en formato BSON.

- Además, se desarrollará una aplicación que permita interactuar con la base de datos mediante operaciones CRUD. De esta manera, los usuarios autorizados podrán registrar, consultar, modificar y eliminar información de acuerdo con las necesidades operativas de la empresa.

Objetivo General

- Diseñar e implementar una solución de base de datos no estructurada para ComercioTech, utilizando MongoDB y una aplicación conectada al DBMS, con el propósito de gestionar de manera segura, eficiente y escalable la información de clientes, productos y pedidos.

Objetivos específicos

- Analizar las necesidades operativas y tecnológicas de la empresa.
- Identificar los requisitos funcionales y no funcionales.
- Seleccionar un sistema operativo compatible con MongoDB.
- Configurar un entorno virtualizado para la instalación del DBMS.
- Instalar y configurar MongoDB aplicando medidas de seguridad.
- Diseñar colecciones para clientes, productos y pedidos.
- Implementar una aplicación conectada a MongoDB.
- Desarrollar operaciones CRUD.
- Poblar la base de datos.
- Documentar las decisiones técnicas y las evidencias de funcionamiento.

Diagnostico e identificación

Diagnóstico de la situación actual

La empresa ComercioTech como hemos mencionado anteriormente, se está expandiendo por lo que ha provocado un aumento en diferentes áreas. Sumándole a esto que existen problemas como el escalamiento, los tiempos de respuesta, dificultad para integrar nuevas funcionalidades y una administración poco eficiente de los datos. La seguridad no es el principal fuerte de la empresa por lo que proteger la información no está dentro del catálogo, ni el control de accesos y el 0 respaldo de datos, exponiéndose a pérdidas totales y exposición de información sensible.

Requisitos de rendimiento

- La solución debe cumplir con los siguientes requisitos:
- Consultas inferiores a 2 segundos.
- Inserción rápida de nuevos documentos.
- Soportar múltiples usuarios simultáneamente.

- Optimizar búsquedas mediante índices.
- Mantener un rendimiento estable aun cuando aumente el volumen de datos.

Requisitos de escalabilidad

- La base de datos deberá permitir:
- Agregar nuevas colecciones sin modificar la estructura existente.
- Incorporar nuevos campos a los documentos cuando sea necesario.
- Aumentar la capacidad de almacenamiento.

Requisitos de disponibilidad

- Para garantizar la continuidad operacional, el sistema deberá:
- Mantener disponibilidad permanente para los usuarios autorizados.
- Contar con mecanismo de respaldo de la información.
- Permitir la recuperación ante fallas.
- Minimizar tiempos de interrupción durante las tareas de mantenimiento.

Requisitos de seguridad

- La solución deberá implementar las siguientes medidas:
- Autenticación de usuarios.
- Control de permisos mediante roles.
- Cifrado de las comunicaciones.
- Respaldo periódico de la base de datos.
- Contraseñas seguras para los usuarios administradores.
- Restricción del acceso únicamente a usuarios autorizados.
- Conclusión del diagnóstico
- El análisis realizado demuestra que la infraestructura actual es débil y no satisface las necesidades de la empresa. Debido al inevitable incremento de información y a la necesidad de disponer de una plataforma flexible y segura, se propone la implementación de MongoDB como sistema gestor de base de datos.
- Su arquitectura orientada a documentos permite almacenar información de forma más flexible que una base de datos relacional tradicional, facilitando la incorporación de nuevos atributos y el crecimiento futuro de la empresa sin afectar significativamente el rendimiento del sistema.

Requisitos del sistema

Requisitos funcionales

Código	Requisito
RF-01	Registrar nuevos clientes.
RF-02	Consultar la información de los clientes.
RF-03	Modificar los datos de un cliente.
RF-04	Eliminar clientes cuando corresponda.
RF-05	Registrar nuevos productos.

RF-06	Consultar el catálogo de productos.
RF-07	Actualizar información y stock de productos.
RF-08	Eliminar productos.
RF-09	Registrar pedidos asociados a un cliente.
RF-10	Asociar uno o más productos a un pedido.
RF-11	Calcular automáticamente el total de un pedido.
RF-12	Consultar el historial de pedidos.
RF-13	Actualizar el estado de un pedido (Pendiente, Enviado, Entregado, Cancelado).
RF-14	Permitir operaciones CRUD mediante una API REST.

Requisitos no funcionales

Código	Requisito
RNF-01	El sistema deberá utilizar MongoDB como gestor de base de datos.
RNF-02	La aplicación deberá desarrollarse con Node.js y Express.
RNF-03	El frontend se desarrollará con React y Vite.
RNF-04	El tiempo de respuesta de las consultas no deberá superar los 2 segundos en condiciones normales.
RNF-05	El sistema deberá validar los datos antes de almacenarlos.
RNF-06	Los correos electrónicos de los clientes deberán ser únicos.
RNF-07	La información deberá almacenarse de forma segura mediante autenticación y control de acceso.
RNF-08	La aplicación deberá ser escalable para soportar el crecimiento del negocio.
RNF-09	El código deberá estar organizado por módulos para facilitar el mantenimiento.
RNF-10	La base de datos deberá contar con al menos 30 registros completos de prueba.

Usuarios del sistema

Usuario	Función
Administrador	Gestiona clientes, productos y pedidos; administra la base de datos.
Vendedor	Registra clientes y crea pedidos.
Bodega	Actualiza el stock y consulta pedidos pendientes.

Restricción del proyecto

- Utilizar MongoDB como base de datos NoSQL.
- Implementar la aplicación mediante una arquitectura cliente-servidor.
- Desarrollar una API REST para la comunicación entre el frontend y la base de datos.
- Utilizar variables de entorno para información sensible.
- Entregar la base de datos con al menos 30 casos completos de prueba.

- Documentar el proceso de instalación y configuración del entorno.

Selección y justificación del Sistema Operativo

Sistema operativo seleccionado

Para el desarrollo de la solución propuesta se utilizó Microsoft Windows 10 como sistema operativo anfitrión, ejecutando Docker Desktop para desplegar MongoDB dentro de un contenedor. Esta configuración permitió aislar la base de datos del sistema operativo, simplificando su instalación, configuración y administración sin afectar el entorno principal del equipo.

Windows 10 fue seleccionado por su compatibilidad con Docker Desktop, Node.js, Express, React y MongoDB Compass, herramientas utilizadas durante el desarrollo del proyecto. El uso de contenedores permitió disponer de un entorno de trabajo estable, reproducible y fácilmente portable, facilitando tanto las pruebas como el despliegue de la aplicación. Además, Docker proporcionó un mecanismo eficiente para gestionar la instancia de MongoDB, permitiendo una rápida puesta en marcha y una administración simplificada de los servicios.

Este enfoque permitió implementar una solución moderna basada en contenedores, manteniendo un entorno organizado, seguro y preparado para futuras ampliaciones del sistema.

Justificación de la elección

La elección de Windows 10 junto con Docker Desktop se fundamenta en los siguientes aspectos:

- Compatibilidad con Docker Desktop y MongoDB Community Edition.
- Facilidad para desplegar MongoDB mediante contenedores sin realizar una instalación nativa.
- Integración con herramientas de desarrollo como Node.js, Express, React, Vite y MongoDB Compass.
- Aislamiento del entorno de ejecución, evitando conflictos con otras aplicaciones instaladas en el sistema.
- Portabilidad del entorno, permitiendo replicar la configuración en otros equipos de manera sencilla.
- Administración simplificada de los servicios mediante contenedores.
- Amplia documentación y soporte por parte de la comunidad de Docker y MongoDB.

Sistema / Plataforma	Ventajas	Desventajas
Windows 10 + Docker Desktop	Fácil instalación, aislamiento mediante contenedores, compatibilidad con Node.js, React, Express y MongoDB, entorno portable y de sencilla administración.	Requiere habilitar la virtualización y WSL2 para ejecutar Docker Desktop.
Ubuntu Server 24.04 LTS	Excelente rendimiento, estabilidad y soporte oficial para MongoDB.	Requiere conocimientos de Linux y una configuración inicial más compleja.
Windows Server	Interfaz familiar para usuarios de Windows y herramientas de administración gráfica.	Mayor consumo de recursos y requiere licencia de pago.
Debian	Muy estable, ligero y con una amplia comunidad de soporte.	Paquetes más conservadores y configuración más manual.

Estas características permitieron construir una plataforma moderna, segura y flexible para el desarrollo de ComercioTech, facilitando tanto las pruebas como la administración de la base de datos durante todo el proyecto.

Configuración del entorno virtualizado

Entorno utilizado

Para el desarrollo, implementación y pruebas de la aplicación se utilizó Docker Desktop como plataforma de contenedores sobre Windows 10. Esta tecnología permitió ejecutar MongoDB dentro de un contenedor independiente del sistema operativo anfitrión, facilitando la configuración del entorno de desarrollo y garantizando una mayor portabilidad de la solución.

El uso de Docker permitió mantener un entorno de ejecución aislado, evitando conflictos entre dependencias y asegurando que la aplicación pudiera reproducirse fácilmente en otros equipos con la misma configuración.

La utilización de contenedores proporcionó las siguientes ventajas:

- Aislamiento del entorno de desarrollo respecto del sistema operativo anfitrión.
- Facilidad para replicar la configuración en distintos equipos.
- Implementación rápida y simplificada de MongoDB mediante contenedores.
- Administración eficiente de los recursos del sistema.
- Mayor estabilidad y seguridad durante las pruebas.
- Portabilidad del entorno de desarrollo sin necesidad de instalar MongoDB de forma nativa.
- Simplificación del despliegue y mantenimiento de la aplicación.

Recursos asignados

Recurso	Configuración
Sistema Operativo	Windows 10 Pro (64 bits)
Virtualización	Docker Desktop (WSL2)
Memoria RAM	4 GB asignados al entorno de desarrollo
Procesador	2 vCPU
Almacenamiento	30 GB disponibles
Base de datos	MongoDB Community Edition (contenedor Docker)
Administrador de BD	MongoDB Compass
Backend	Node.js + Express
Frontend	React + Vite
ORM / ODM	Mongoose
Gestor de paquetes	npm
Control de versiones	Git + GitHub

Configuración realizada

Durante la preparación del entorno de desarrollo se realizaron las siguientes configuraciones para garantizar el correcto funcionamiento de la aplicación y la base de datos:

- Instalación y configuración de Docker Desktop sobre Windows 10.
- Habilitación de WSL2 y la virtualización para permitir la ejecución de contenedores.
- Descarga de la imagen oficial de MongoDB Community Edition desde Docker Hub.
- Creación y configuración del contenedor de MongoDB con credenciales de acceso y asignación del puerto correspondiente.
- Configuración de la conexión a la base de datos mediante variables de entorno (.env).
- Instalación de Node.js y del gestor de paquetes npm.
- Instalación de las dependencias del proyecto para el backend y el frontend.
- Configuración de MongoDB Compass para la administración y verificación de la base de datos.
- Verificación de la conectividad entre el backend desarrollado con Node.js y la instancia de MongoDB ejecutándose en Docker.

Ejecución de pruebas de funcionamiento para validar las operaciones CRUD y la correcta persistencia de la información en la base de datos.

Instalación y configuración de MongoDB

Instalación de MongoDB

Para la implementación de la base de datos se utilizó MongoDB Community Edition, desplegado mediante un contenedor Docker utilizando la imagen oficial disponible en Docker Hub. Esta metodología permitió disponer de un entorno de ejecución aislado, facilitando la instalación, configuración y administración de la base de datos sin necesidad de realizar una instalación nativa sobre el sistema operativo.

El contenedor fue configurado con un usuario administrador, contraseña de acceso y la asignación del puerto correspondiente para permitir la comunicación con el backend desarrollado en Node.js y Express. Posteriormente, se verificó el correcto funcionamiento del contenedor mediante Docker Desktop y el comando `docker ps`, confirmando que la instancia de MongoDB se encontraba en ejecución.

Finalmente, se comprobó la conectividad utilizando MongoDB Compass y la aplicación backend, verificando que la base de datos aceptaba conexiones correctamente y permitía ejecutar operaciones de creación, consulta, actualización y eliminación (CRUD) sobre las colecciones del sistema. Esta validación confirmó que la infraestructura estaba correctamente configurada y lista para su utilización durante el desarrollo del proyecto.

Diseño de la Base de Datos

Modelo documental

La solución fue desarrollada utilizando un modelo documental propio de MongoDB. En lugar de utilizar tablas relacionadas, la información se organiza mediante colecciones compuestas por documentos BSON.

Este enfoque proporciona mayor flexibilidad para almacenar información con estructuras variables y facilita el crecimiento futuro del sistema.

Las principales colecciones implementadas fueron:

- Customers
- Products
- Orders

Colección Customers

La colección Customers almacena la información de los clientes registrados en el sistema.

Campos principales:

- Nombre
- Apellido
- Correo electrónico
- Teléfono
- Dirección
- Estado
- Fecha de registro

Cada cliente posee un identificador único generado automáticamente por MongoDB.

Colección Products

La colección Products contiene el catálogo de productos disponibles para la venta.

Campos principales:

- Nombre
- Descripción
- Categoría
- Precio
- Stock
- SKU
- Estado

El stock puede actualizarse automáticamente cuando se registra un nuevo pedido.

Colección Orders

La colección Orders almacena todos los pedidos realizados por los clientes.

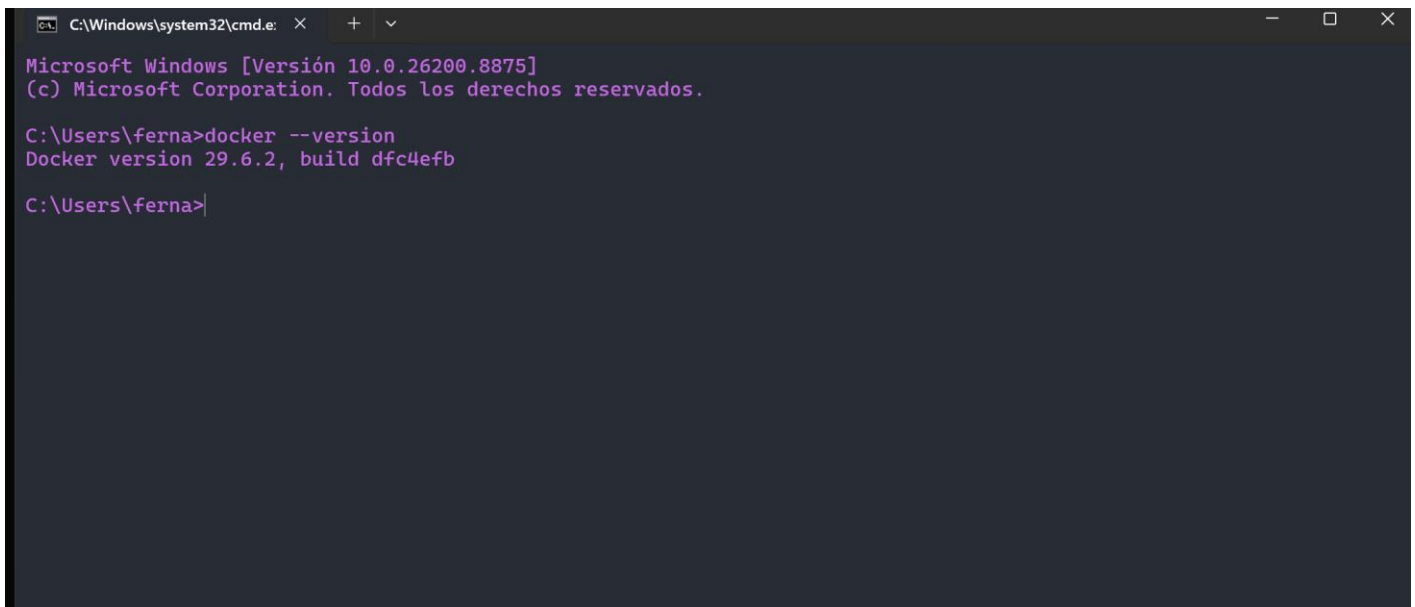
Cada documento incluye:

- Cliente asociado
- Lista de productos
- Cantidad
- Precio unitario
- Subtotal

- Total
- Estado del pedido
- Fecha de creación

La relación entre pedidos, clientes y productos se implementó mediante referencias utilizando ObjectId.

Evidencias



```
C:\Windows\system32\cmd.e:  X  +  v
Microsoft Windows [Versión 10.0.26200.8875]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\ferna>docker --version
Docker version 29.6.2, build dfc4efb

C:\Users\ferna>
```

Figura 1. Verificación de la instalación de Docker Desktop.

```
Storage Driver: overlayfs
 driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: cgroupfs
Cgroup Version: 2
Plugins:
 Volume: local
 Network: bridge host ipvlan macvlan null overlay
 Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
CDI spec directories:
 /etc/cdi
 /var/run/cdi
Discovered Devices:
 cdi: docker.com/gpu=webgpu
Swarm: inactive
Runtimes: io.containerd.runc.v2 nvidia runc
Default Runtime: runc
Init Binary: docker-init
containerd version: e53c7c1516c3b2b98eb76f1f4117477e6f4e66
runc version: v1.3.6-0-g491b69ba
init version: de40ad0
Security Options:
 seccomp
  Profile: builtin
 cgroupns
Kernel Version: 6.18.33.2-microsoft-standard-WSL2
Operating System: Docker Desktop
OSType: linux
Architecture: x86_64
CPUs: 16
Total Memory: 7.443GiB
Name: docker-desktop
ID: 8583914d-52be-464f-92aa-b96f45304db5
Docker Root Dir: /var/lib/docker
Debug Mode: false
HTTP Proxy: http.docker.internal:3128
HTTPS Proxy: http.docker.internal:3128
No Proxy: hubproxy.docker.internal
Labels:
 com.docker.desktop.address=npipe://\\.\pipe\docker_cli
Experimental: false
Insecure Registries:
 hubproxy.docker.internal:5555
 127.0.0.0/8
 ::1/128
Live Restore Enabled: false
Firewall Backend: iptables

C:\Users\ferna>
```

Figura 2. Verificación del motor Docker Desktop.

127.0.0.1:27018

> Open MongoDB shell

+ Create database

Refresh

Database name 	Storage size 	Data size 	Collections 	Indexes 
admin	40.96 kB	583.00 B	2	3
config	4.10 kB	0 B	1	2 
local	20.48 kB	13.27 kB	1	1

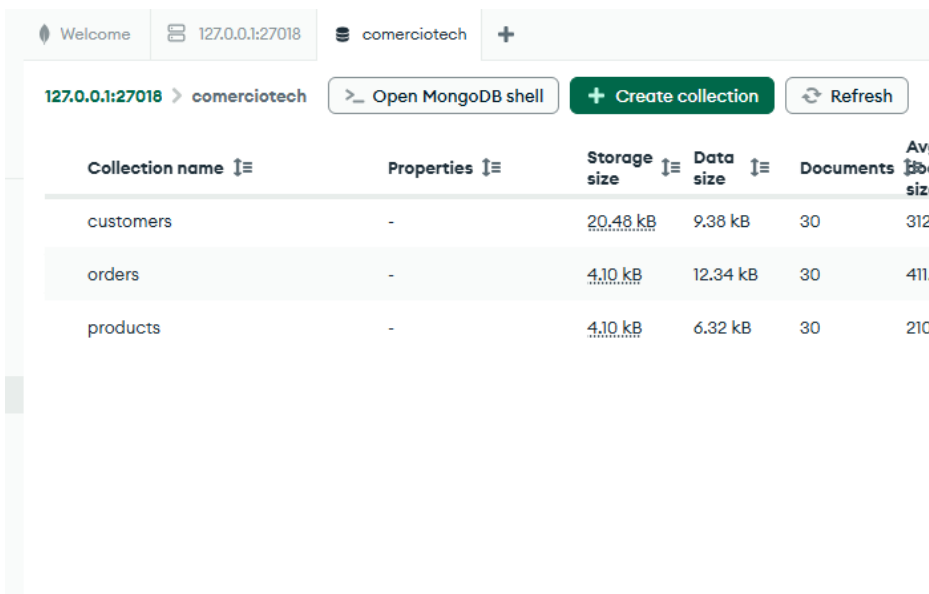
Figura 3. Conexión exitosa a MongoDB Compass mediante un contenedor Docker configurado en el puerto 27018.


```
PS C:\Users\ferna\OneDrive\Documentos\Inacap\Base de Datos No Es
estructuradas\comerciotech\backend> npm run dev

> backend@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
◇ injected env (2) from .env // tip: ↘ auth for agents [www.vest
auth.com]
MongoDB connected successfully
Server running on port 5000
```

Figura 4. Conexión del backend de ComercioTech con MongoDB desplegado mediante Docker.



Collection name	Properties	Storage size	Data size	Documents	Average document size
customers	-	20.48 kB	9.38 kB	30	312
orders	-	4.10 kB	12.34 kB	30	411
products	-	4.10 kB	6.32 kB	30	210

Figura 5. Base de datos ComercioTech y sus colecciones visualizadas en MongoDB Compass.

```
C:\Users\ferna\OneDrive\Documentos\Inacap\Base de Datos No Estructuradas\comerciotech\backend>npm run dev
```

```
> backend@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
◇ injected env (2) from .env // tip: % suppress logs { quiet: true }
MongoDB connected successfully
Server running on port 5000
```

Figura 6. Inicio del servidor backend y conexión exitosa con la base de datos MongoDB desplegada en Docker.

[illegible]

Figura 7. Respuesta de la API REST obtenida desde el navegador.

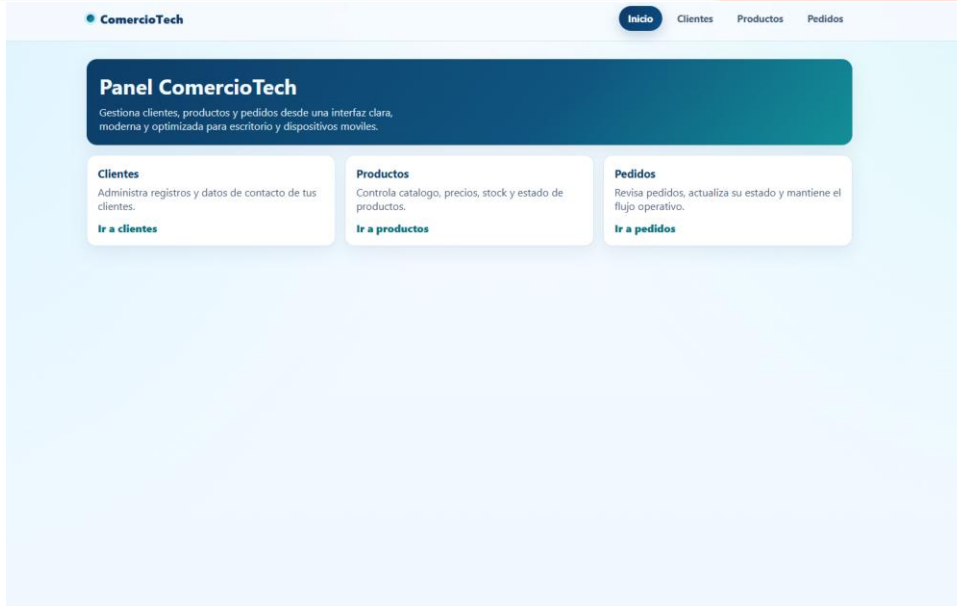


Figura 8. Interfaz principal de la aplicación ComercioTech desarrollada con React y Vite.

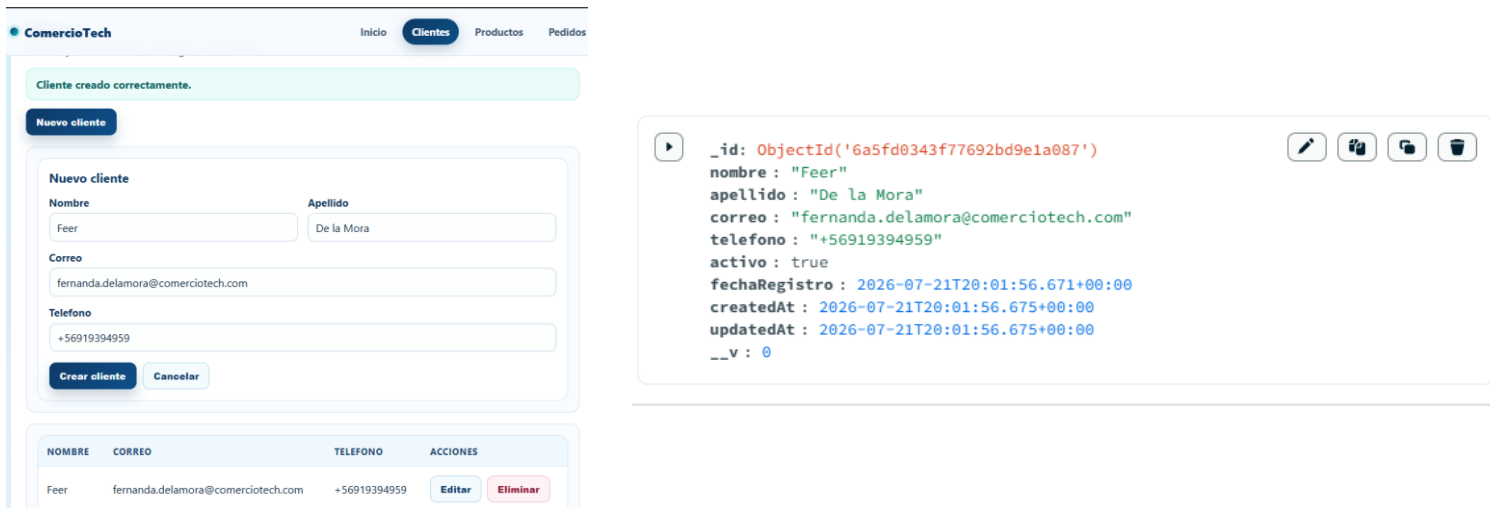


Figura 9. Colecciones creadas automáticamente en la base de datos ComercioTech tras realizar operaciones CRUD desde la aplicación.



APRENDER HACIENDO A LO LARGO DE LA VIDA



INSTITUTO PROFESIONAL INACAP

ACREDITADO • NIVEL DE EXCELENCIA

- Gestión Institucional • Docencia de Pregrado
- Vinculación con el Medio
- Próxima Acreditación Septiembre 2030

www.inacap.cl